① How would you design a scalable Playwright framework?

Answer:

A scalable Playwright framework should include:

- Page Object Model (POM)
- Fixtures for setup and teardown
- API utilities
- Environment configurations
- Reporting layer
- CI/CD integration
- Parallel execution

Key Point:

> Focus on maintainability, reusability, and scalability.

---

② How does Playwright handle auto-waiting?

Answer:

Playwright automatically waits for elements to become:

- Visible
- Stable
- Enabled
- Ready to receive events

Example:

```
await page.locator('#submit').click();
```

Benefit:

> No need for most explicit waits.

3 What is the difference between locator() and page.$()?

Answer:

locator()	page.$()
• Supports auto-waiting • Automatically retries • Re-resolves elements before every action • More stable • Recommended approach	• Returns immediately • No auto-waiting • No retry mechanism • More prone to flaky tests



Key Point:
Always prefer locator() over page.$() for reliable and stable tests.

---

4 How would you reduce flaky tests?

Answer:

Best practices include:


• Use stable and unique locators


• Avoid waitForTimeout()



• Mock external APIs


• Configure retries in CI


• Capture traces and videos for debugging


• Maintain proper test data



Key Point:
Good practices + proper debugging = stable and reliable tests.

5. Explain Playwright Fixtures.

Answer:

Fixtures provide reusable test setup and context, making tests cleaner and easier to maintain.

Example:

```
import { test } from '@playwright/test';

export const test = base.extend({
   loginPage: async ({ page }, use) => {
      await use(new LoginPage(page));
   }
});
```

Benefits:

- Dependency injection
- Cleaner tests
- Reusable setup
- Better maintainability

---

6. How do Browser Contexts improve execution?

Answer:

A Browser Context is an isolated browser session.

Example:

```
const context = await browser.newContext();
const page = await context.newPage();
```



Key Point:
Each browser context has its own session, cookies, and permissions.

Benefits:


Faster execution


Isolated cookies and storage


Parallel execution


Supports multi-user testing


Better stability

7. How would you implement authentication once and reuse it across tests?

Answer:

Save the authentication state once and reuse it in all tests.

Store authentication:

```
await page.context().storageState({
    path: 'auth.json'
});
```

Reuse authentication:

```
test.use({
    storageState: 'auth.json'
});
```

Benefits:

- Avoid repeated login
- Faster execution
- Less server load
- Consistent sessions

8. How do you handle network mocking in Playwright?

Answer:

Use page.route() to intercept network requests.

Example:

```
await page.route('**/users', route => {
   route.fulfill({
      status: 200,
      body: JSON.stringify({ name: 'Tanu' })
   });
});
```

Use Cases:

- Mock third-party APIs
- Negative testing
- Test edge cases
- Performance testing
- Remove external dependencies

9. What is the difference between browser.newPage() and context.newPage()?

Answer:

browser.newPage()	context.newPage()
⊗ Creates a page in the default context	✔ Creates a page in a specific context
⊗ Less isolation	✔ Better isolation
⊗ Shares cookies and storage	✔ Independent cookies and storage
⊗ Not recommended for testing	✔ Recommended approach

Example:

```
const context = await browser.newContext();

const page = await context.newPage();
```

Benefits:

- Better session isolation
- Cookie management
- Permission handling
- Supports parallel execution

---

10. How does Playwright support parallel execution safely?

Answer:

Each test runs in its own browser context.

Playwright ensures:

- ✔ Separate browser context
- ✔ Separate session and storage
- ✔ Test isolation
- ✔ Reduced interference
- ✔ Better performance

Example:

```
export default defineConfig({
  workers: 4
});
```

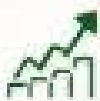
Benefits:



Faster execution



Better resource utilization



Scalable automation



Reliable tests

11. How would you integrate Playwright into CI/CD?

Answer:

Typical pipeline:

1. Install dependencies
2. Install Playwright browsers
3. Execute tests
4. Publish reports
5. Archive artifacts

Example GitHub Actions:

```yaml
- run: npm ci
- run: npx playwright install
- run: npx playwright test
- uses: actions/upload-artifact@v4
  with:
    name: playwright-report
    path: playwright-report/
```

Important Points:

- Headless execution
- Retries in CI
- Parallel execution
- Artifact retention
- Fail-fast strategy

---

12. How do you capture logs, screenshots, and traces for debugging?

Screenshot

```ts
await page.screenshot({
  path: 'screenshot.png',
  fullPage: true
});
```

↓

Used for visual debugging.

Trace

```ts
export default defineConfig({
  use: {
    trace: 'on-first-retry'
  }
});
```

↓

Used for step-by-step debugging.

Video

```ts
export default defineConfig({
  use: {
    video: 'retain-on-failure'
  }
});
```

↓

Used to reproduce failures.

☆ Key Takeaway: Screenshots, Traces, and Videos help in quick issue identification and reduce debugging time.

13. Explain API testing capabilities in Playwright.

Answer:

Playwright supports API testing alongside UI testing.

Example:

```
const { request } = await playwright.request.newContext();

const response = await request.get('/users');

expect(response.ok()).toBeTruthy();
```

Benefits:

- UI and API in one framework
- Token sharing
- End-to-end validation
- Faster execution

---

14. How do you handle dynamic locators in Playwright?

Answer:

Use locator() with filters.

Example:

```
const user = 'Tanu';

const card = page
      .locator('.user-card')
      .filter({ hasText: user });

await card.click();
```

Best Practices:

- Use locator filters
- Avoid XPath where possible
- Prefer Role, Text, and Test ID selectors
- Keep locators maintainable

15. Explain the Playwright retry mechanism.

Answer:

Configure retries in playwright.config.ts.

Example:

```
retries: 2
```

Best Practices:

- Don't use retries to hide failures
- Enable retries mainly in CI
- Analyze flaky tests
- Quarantine unstable tests

---

16. How would you optimize Playwright execution time?

Answer:

Optimization techniques:

1. Parallel execution
2. Reuse authentication state
3. API-based setup
4. Headless execution
5. Reduce dependencies
6. Run tagged tests

Example:

```
npx playwright test --grep @smoke
```

Advanced techniques:

- Test sharding
- Docker execution
- Browser reuse

17. How do you validate downloads in Playwright?

Answer:

Example:

```
const downloadPromise = page.waitForEvent('download');

await page.click('#download');

const download = await downloadPromise;
```

Validation includes:

- File name
- File type
- File content

---

18. How would you test multi-tab or popup scenarios?

Answer:
Use Promise.all() with context listeners.

Example:

```
const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    page.click('#open')
]);
```

Important:

- Always use context-level event listeners for reliable popup handling.

19. Explain Playwright hooks and best practices.

Answer:

Common hooks:

- beforeAll
- beforeEach
- afterEach
- afterAll

Best Practices:

- Keep hooks lightweight
- Avoid test dependencies
- Use hooks only for setup and cleanup
- Properly release resources

---

20. How would you design a Hybrid UI + API automation strategy?

Answer:

Use APIs for:

- Test setup
- Cleanup
- Test data creation
- Preconditions

Use UI only for:

- Business validation

Benefits:

- Faster execution
- Fewer flaky tests
- Stable environments
- Lower maintenance

Quick Commands

Task	Command
Run all tests	npx playwright test
Run a specific file	npx playwright test login.spec.ts
Run with UI	npx playwright test --ui
Run in headed mode	npx playwright test --headed
Debug tests	npx playwright test --debug
View report	npx playwright show-report